

Lab Assignment 2: Sum the File

Purpose:

The purpose of this assignment was to read and parse through a set of data, specifically integers, from a file in order to produce and print the sum of the data on the screen. The data set includes the maximum number of 1000 integers in the file.

Approach:

I began this project by planning the basic flow of the program, such as loading and reading the contents of the file, storing the current string (data number from the file) into a buffer, processing the string to its integer value, adding number to the total sum, and printing out the results from the provided data. For the “.data” section, the “pathname” was defined to open the file with the specified name, “newline” for the newline character, and the output message prompt. For the “.bss” section, there is reserved space for each individual element/variable, such as the buffer, sum, current number, and ASCII representation of the sum (to facilitate printing the result onto the screen after all data points in the specified file were parsed, processed, and added to the sum value). For the “.text” section, it contains the actual code to perform the summation and output of the result.

At the beginning of the program, the pathname is set to the file containing the integers (represented by strings), such as “randomInt100.txt” to retrieve the data points that will be summed. In addition, there is space reserved for the buffer to store a read-in data point (“buffer”), total sum of all the integers (“sum”), and current number (“currNum”) as the read-in characters/digits/integers for the data point. Next, the current number is loaded into a buffer within the esi register to process it. This process demonstrated in the “process_data_loop” function includes verifying each character through conditional checks, such as comparing the delimiter values as done in the “check_delimiter” and comparing it with null to denote the end of the buffer; if the end of the string is reached or the end of the file, then it will jump to the “exit” subroutine of the program. However, if the character is a digit, then its ASCII representation of the character is converted to its integer representation where the integer of the current number is obtained by multiplying it by 10 to shift left by one decimal place as denoted as “imul ebx, ebx, 10” and it will add the new digit within the eax register to the current number, updating the processed digits back into the stored current number. Then, the current number is added to the sum and resetting the current number to 0 in order to process the next number in the data point as shown in the “next_character” function. This process of “process_data_loop” repeats until the end of the file is reached and all the data integers have been accounted for. After this process is finished, the “exit” routine will handle the last data number by parsing through it and

adding it to the sum of all data integers. Finally, the sum result is converted to its string representation to print the total sum onto the screen and the sum is displayed.

Code:

```
; Name: Amrita Shrestha  
; Lab Assignment 2 - Sum the File  
; Lab Section: 03  
; Date: 4/3/25
```

section .data

```
pathname db "randomInt100.txt", 0    ; Establishes the path to the text file  
  
; path name to other custom tester data files  
; pathname db "sumTest1.txt", 0      ; Establishes the path to the custom text file  
; pathname db "sumTest2.txt", 0      ; Establishes the path to the custom text file  
  
msg db "The sum is: ", 0x0A          ; Output message for sum result  
len equ $ - msg  
  
newline db 10, 0                     ; Newline character
```

section .bss

```
buffer resb 1000                     ; Stores content from the file in buffer  
sum resd 1                           ; Stores the total sum of all the ints in file  
currNum resd 1                       ; Current data point  
sumStr resb 12                       ; ASCII representation of sum result
```

section .text

```
global _start
```

```
_start:
```

```
; Opens the specified file  
mov eax, 5                           ; System open  
mov ebx, pathname                     ; Stores file name in ebx  
mov ecx, 0                           ; Reads file  
int 0x80                             ; Interrupt
```

```
mov ebx, eax
```

```
; Reads file content into the buffer
```

```

mov eax, 3                ; System call
mov ecx, buffer
mov edx, 1000
int 0x80

mov edx, eax              ; Reads the byte

; Initialize sum and currNum to store total sum result and current data point
mov dword [sum], 0
mov dword [currNum], 0
mov esi, buffer

; Parses each data point in file
process_data_loop:
    mov al, [esi]          ; Loads byte in esi into al
    cmp al, 0              ; Compares character with null
    je exit                ; If null, jump to exit
    cmp al, '0'
    jl check_delimiter     ; Checks for delimiter
    cmp al, '9'
    jg check_delimiter     ; Checks for delimiter

    sub al, '0'            ; Converts ASCII character to int
    movzx eax, al
    mov ebx, [currNum]      ; Loads current data point to ebx
    imul ebx, ebx, 10       ; Multiplies currNum by 10 (shift left by one decimal place)
    add ebx, eax            ; Adds the new digit to current number
    mov [currNum], ebx      ; Stores updated number back into currNum

    jmp next_character      ; Jumps to next character

; Checks for a space to indicate end of a number
check_delimiter:
    mov eax, [currNum]
    add [sum], eax          ; Adds current number to the total sum
    mov dword [currNum], 0 ; Reset num to 0 for the next number

next_character:
    inc esi                 ; Increments esi to point to the next character in the buffer

```

```
    jmp process_data_loop          ; Processes the next digit until each digit is processed for  
currNum
```

```
; Stores the total sum and displays the result  
exit:
```

```
    mov eax, [currNum]  
    add [sum], eax
```

```
; Converts sum to its ASCII representation
```

```
    mov eax, [sum]
```

```
    mov edi, sumStr
```

```
    call int_to_str                ; Converts the number to a string
```

```
; Outputs the sum onto the terminal
```

```
    mov eax, 4                    ; system call number (sys_write)
```

```
    mov ebx, 1                    ; file descriptor (stdout)
```

```
    mov ecx, msg                  ; message to write
```

```
    mov edx, len                  ; length of output message
```

```
    int 0x80                      ; calls kernel
```

```
; Outputs the sum
```

```
    mov eax, 4                    ; system call number (sys_write)
```

```
    mov ebx, 1                    ; file descriptor (stdout)
```

```
    mov ecx, sumStr               ; pointer to sum result
```

```
    mov edx, 12                   ; length of sum result
```

```
    int 0x80                      ; calls kernel
```

```
; Print newline
```

```
    mov eax, 4
```

```
    mov ebx, 1
```

```
    mov ecx, newline
```

```
    mov edx, 1
```

```
    int 0x80
```

```
; Outputs the newline
```

```
    mov eax, 4                    ; system call number (sys_write)
```

```
    mov ebx, 1                    ; file descriptor (stdout)
```

```
    mov ecx, newline              ; pointer to newline character
```

```
    mov edx, 1                    ; length of newline character
```

```
    int 0x80                      ; calls kernel
```

```

; Exit the program
mov eax, 1                ; system call to exit
xor ebx, ebx              ; Exit code 0
int 0x80                  ; calls kernel

; Converts the int value of sum back to its string representation
int_to_str:
    mov ecx, 10
    mov ebx, 0
    mov edi, sumStr + 11
    mov byte [edi], 0      ; Null-terminate string
    dec edi                ; Move edi one step back to store digits

convert:
    mov edx, 0
    div ecx
    add dl, '0'
    mov [edi], dl
    dec edi
    inc ebx
    test eax, eax
    jnz convert

    inc edi
    ret

```

Outputs:

Note: Each data file were tested separately by commenting/uncommenting the lines corresponding to each test case (containing one data file) in the code

```

[amritas2@linux6 assign2]$ nasm -f elf32 -g sum.asm
[amritas2@linux6 assign2]$ ld -m elf_i386 -o sum sum.o
[amritas2@linux6 assign2]$ ./sum
The sum is:
4679

```

Sum for “randomInt100.txt” (test case 1 - provided in project description)

```
[amritas2@linux6 assign2]$ nasm -f elf32 -g sum.asm
[amritas2@linux6 assign2]$ ld -m elf_i386 -o sum sum.o
[amritas2@linux6 assign2]$ ./sum
The sum is:
825
```

Sum for “sumTest1.txt” (test case 2 - custom data file 1)

```
[amritas2@linux6 assign2]$ nasm -f elf32 -g sum.asm
[amritas2@linux6 assign2]$ ld -m elf_i386 -o sum sum.o
[amritas2@linux6 assign2]$ ./sum
The sum is:
1868
```

Sum for “sumTest2.txt” (test case 3 - custom data file 2)